The UNIX Time Sharing System

1/7
ECS 251
Amanda Raybuck

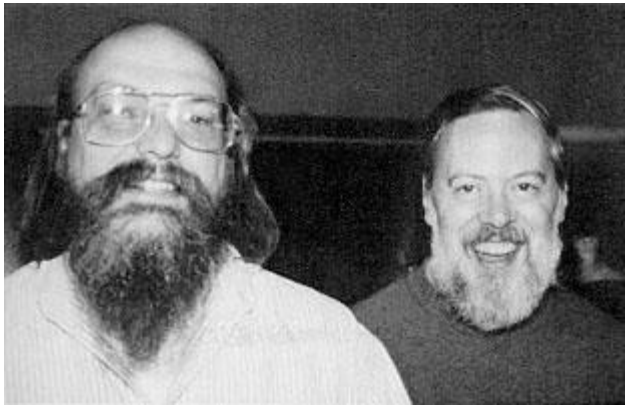
Slides adapted from course materials from Mosharaf Chowdhury and Hakim Weatherspoon

Administrivia

- Pick a paper to present by Friday

Background on Thompson and Ritchie

- Worked at Bell Labs, helped author the Multics OS
  - Multics influenced many aspects of UNIX
  - But underlying design philosophy was quite different
  - Ken Thompson, 2007:
    - Multics was *“overdesigned and overbuilt and over everything. It was close to unusable”*
- Ritchie: Key developer of UNIX and the C programming language
- Thompson: Key developer of UNIX, Plan 9, QED, ed, UTF-8



Both earned **Turing Awards** for UNIX in 1983

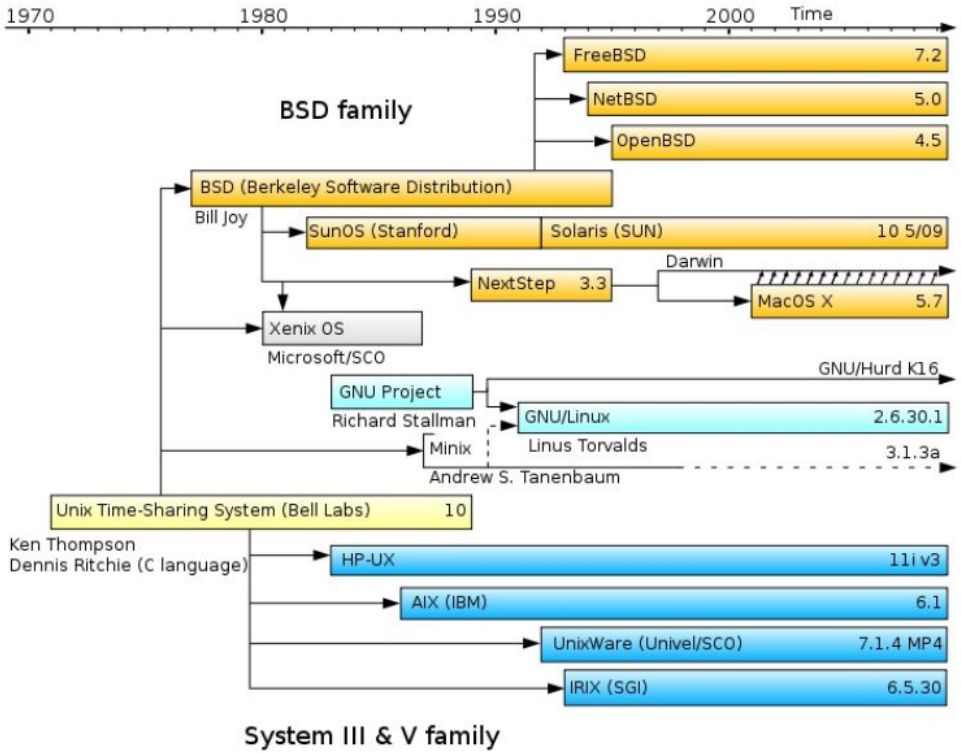
What Did Computers Look Like in 1974?

- 16-bit word (8-bit byte)
- 144K bytes core memory
  - UNIX took 42K bytes
  - Minimal system 50K bytes
- Storage
  - 1M byte fixed-head disk
  - 4X 2.5M byte moving-head disk cartridge
  - 1X 40M byte moving-head disk pack
- 14 variable speed communication interfaces
- Many other attached devices
- **Batch Processing**: Jobs processed from queues one at a time
- **Time Sharing**: Many users could use the system at once, *interactive*

The PDP-11

The UNIX Family Tree

- Unix was highly influential for OS design and development
- Many familiar OSs today can trace back to UNIX

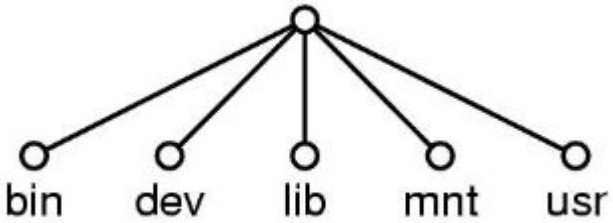


Key Ideas

- Elegant combination of a few concepts that fit together well
- Small, general API
- Simple!
  - Abstractions - “everything is a file”
  - File system design - “hierarchical”
  - Connectors - “pipes”
  - Maintenance - “self-maintained”

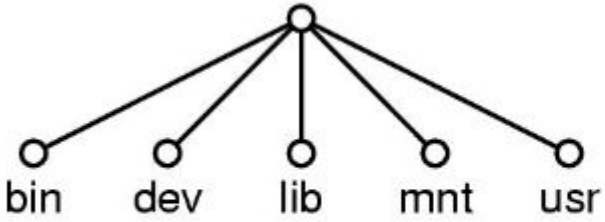
Everything is a File

- **Regular files** are files - contain raw data
  - UNIX itself does not impose any structure to files, but applications may
- **Directories** are files - “files of files”
  - Contain pointers to files in a hierarchical format
  - UNIX controls the content of directories
- **Devices and networks** are files - “special files”
  - Read/written just like normal files, but result in activation of a device

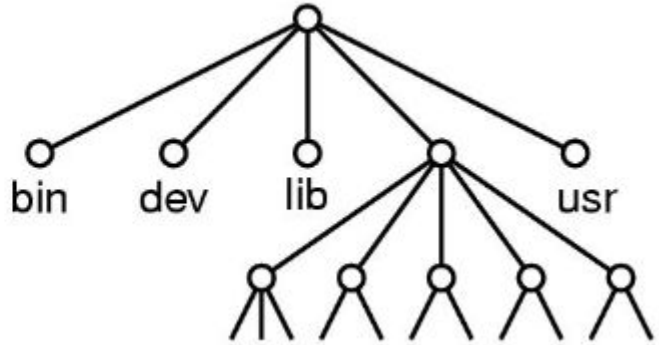


Mounting

- Allows for handling of removable file systems
- Takes advantage of the hierarchical tree structure of the FS
  - Mount replaces a leaf of the tree with the root of the mounted FS
  - (Mostly) no distinction between files on the permanent FS or the mounted FS



Before mount



After mount

Uniform IO Model

- Open, close, read, write, seek
  - Uniform calls eliminates differences between devices
  - Two categories of files: character (or byte) stream and block I/O, typically 512 bytes per block
  - IO appears synchronous and unbuffered - read and write return immediately
    - But IO may be buffered in memory
- Other system calls
  - status, chmod, mkdir, ln
- One way to “talk to the device” more directly:
  - ioctl, a grab-bag of special functionality
- **Advantage:** Uniform naming and protection model
  - File and device I/O are as similar as possible
  - File and device names have the same syntax and meaning, can pass as arguments to programs
  - Same protection mechanism as regular files

Protection and Access Control

- Protection follows the hierarchical model of the FS
- 2 sets of RWX bits determine permissions of owner of file and everyone else
- set-user-id bit
  - Allows the userID to temporarily change to the owner of a file
  - Allows for privileged programs to use files inaccessible to other users
  - Actual userID is still available for permission checks
- Super-user: special userID that is exempt from regular file access constraints

Processes and Images

- **Image**: An execution environment
  - The core image, general register values, status of open files, current directory
- **Process**: The execution of an image
- Process creation: fork()
  - Creates a new child process that is a clone of the parent
  - Independent copies of the core image, share any open files
- Process communication: pipes! Look/act just like files
- Execute, wait, and exit

An Aside: Is Fork() Good?

A Fork() in the Road by Baumann et al.

- Fork() actually hides a lot of complexity that makes supporting fork() a pain
  - 25 (!!) special-cases in how parent state is copied to child
  - Fork() does not compose and is not thread safe
  - Fork() is insecure - up to the programmer to remove any unneeded state of parent from child
  - Fork() is slow and does not scale
- We should deprecate fork() and work on improving the alternative APIs
  - A spawn API, clone API, use threads instead, Copy-on-Write memory

Fork is an anachronism: a relic from another era that is out of place in modern systems where it has a pernicious and detrimental impact. As a community, our familiarity with fork can blind us to its faults.

- Baumann et al.

The Shell

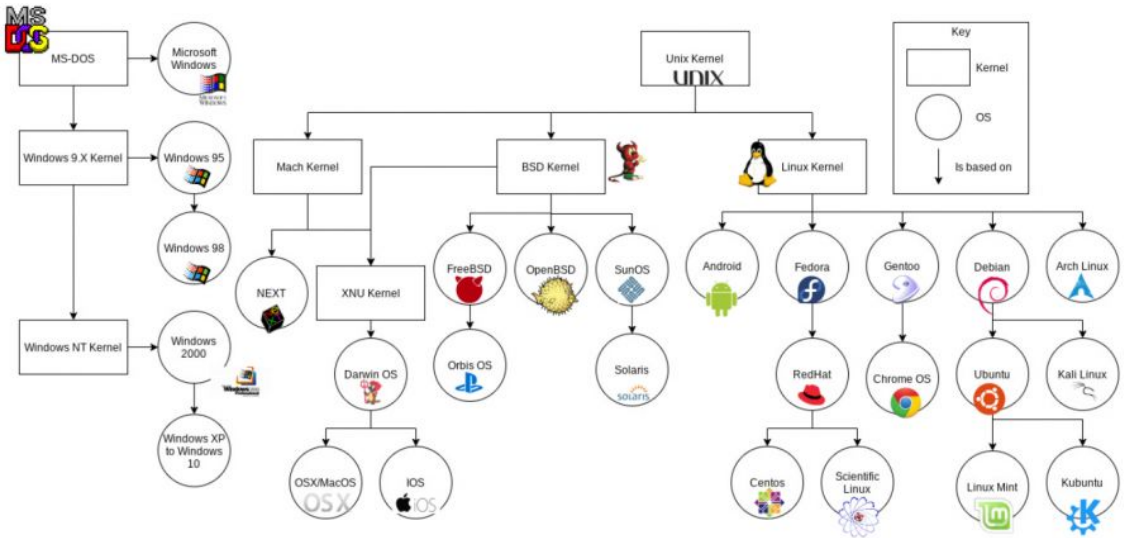
- A very powerful and useful program that runs in userspace
- Command structure: cmd arg1 … argn
  - cmd is the name of a file, arguments are passed from the shell to the command
- Supports I/O redirection with “<” and “>”
  - Allows the creation of pipelines where the output of one command is the input of the next command (e.g., cat file.txt | grep foo | wc -l)
- Allows for scripts and background processing

UNIX Perspective

- Not designed to meet any predefined objective
- Goal: A comfortable environment to explore ideas and inventions in OS
- Design considerations:
  - Programmer convenience: By programers, for programmers
  - Economy and elegance of design to counter size constraints
  - Self-maintaining: Designers of the system used the system themselves

The Staying Power of UNIX

- Linux is far more comprehensive but the core simplicity of the UNIX API remains
- Simplicity in design remains an important goal for OS developers
  - Keeps us from over-specializing the system in every way, hard to understand



source: computersciencewiki.org

Discussion

- Is the “everything is a file” abstraction a good/useful abstraction?
- Is the fork() system call still a good/useful API?
- What aspects described in the paper are still around today? Have any changed? Why?
- Do we like simple, uniform APIs? Are there instances where more specialized APIs are desirable?

Next Time

- State preference for leading a lecture paper
- The ExoKernel paper